Between the Issue and the Warning: Tracing Security-Relevant Decisions in Agent-Associated Pull Requests

ARNO STRAUSS, School of Computer Science and Applied Mathematics, University of the Witwatersrand, South Africa

A GitHub issue can specify a task without prescribing how to carry it out. When a security scanner reports a condition in the resulting pull request, neither the issue nor the warning alone explains the choice of implementation. To examine that gap, this study followed 3,809 selected issue–PR workflows with public coding-agent indicators. Scanner findings were traced to 455 changed operations, grouped into 342 implementation strategies. In the original 122-workflow annotation group, dependencies accounted for much of the predominance of supporting changes; among source-code warnings, direct and supporting changes were equally common. Earlier repository code, explicit guidance and tooling behaviour sometimes offer plausible explanations, although the implementer's private reasoning remains unknown. Matched issues differed in instruction specificity. Across the analyses, the task's stated goal, the mechanism chosen to implement it and the scanner's reported condition emerge as distinct questions.

CCS Concepts: • **Software and its engineering** → **Software creation and management**; • **Security and privacy** → *Software and application security*.

Additional Key Words and Phrases: Empirical software engineering, coding agents, GitHub issues, static analysis, pull requests, implementation provenance

2026 BSc Honours research paper. Supervisors: Prof. Michael Pradel (CISPA) and Prof. Hima Vadapalli (Wits).

1 Introduction

A GitHub issue requested Spotify playback in a music bot. The linked pull request (PR) added the feature and an authentication verifier. In the verifier, random bytes were mapped to an alphabet using a modulo operation. CodeQL flagged the resulting token distribution as biased, although a different token-generation idiom already existed in the repository. The request, new operation and earlier example are visible. Why that operation was chosen is not.

A request may establish the purpose of a change yet leave its algorithm, dependency or validation check open. A scanner identifies a condition in the patch, not the evidence used to select that implementation.

Existing studies show how agents resolve repository-level issues and how implementation depends on access to code and tools [5, 13, 16]. Other studies demonstrate that deliberately crafted bug reports or issues can induce harmful changes [1, 9]. In those experiments, the adversarial objective is known. A public development record, by contrast, may reveal neither the issue author's intent nor the agent's private instructions and subsequent human edits. The initial search for malicious issues could not establish intent from these records. Accordingly, this paper examines observable changes rather than treating scanner findings as proof of attack or autonomous error.

The main analysis follows scanner reports back to changed operations and asks whether the issue required those operations or merely the broader task. A matched comparison and selected replays address narrower questions about issue wording and available context.

Author's Contact Information: Arno Strauss, School of Computer Science and Applied Mathematics, University of the Witwatersrand, Johannesburg, South Africa, 2613224@students.wits.ac.za.

1.1 Research Questions and Hypothesis

The exploratory hypothesis is that an issue can specify the desired behaviour without prescribing the operation later flagged by a scanner. It concerns what the public record shows, not what an agent privately considered. Three questions guide the analysis:

RQ1. How do warning-associated changes relate to the requested task, and what can the public record explain about their implementation?

RQ2. How do issue descriptions and other observable characteristics differ between workflows with scanner findings and comparable workflows without them?

RQ3. What happens in selected task replays when additional context or information derived from a historical warning is supplied?

1.2 Contribution and Scope

The analysis traces scanner findings to changed operations, examines their role in the requested task and identifies what the public record can explain about their implementation. From 3,809 selected merged issue–PR workflows with coding-agent indicators, the study traces findings to 455 changed operations and 342 grouped implementation strategies. Matched comparisons and retrospective replays provide supporting observations rather than causal or prevention claims.

2 Background

2.1 Tasks, Patches and Coding Agents

Issues record requests, bugs and discussion; PRs contain the proposed code changes. Linking the two permits the request to be studied alongside a patch, although the link does not prove that the issue initiated or dictated the work [2]. Coding agents can also consult repository instructions, retrieve files and respond to tool feedback beyond what the public issue contains [11, 13].

The label *agent-associated* reflects public indicators such as a PR author, branch pattern or reference to an agent tool in issue or PR text. Their evidential strength varies: mentioning an agent in an issue, for example, does not establish who wrote the subsequent patch. The cohort therefore should not be read as a census of autonomously written PRs.

2.2 What a Security Warning Means

Code-analysis tools look for patterns or program properties, whereas dependency scanners compare package versions with advisories and secret scanners flag credential-like material. Their findings concern different kinds of evidence and should not be interpreted as interchangeable security defects. In this paper, *scanner-positive* refers to a workflow with a retained finding classified as introduced by its PR under the recorded scanning and attribution procedure. It does not mean that exploitability has been demonstrated.

A scanner cannot determine whether the requested feature was delivered. Tests and syntax checks provide only partial evidence of functionality, and a patch without recorded alerts may still be incomplete [1]. This matters for the task replays, where outputs differ in size as well as scanner results.

2.3 Task Relevance and Mechanism Choice

Task relation concerns the role of a change: delivering requested behaviour directly or supporting it through tests, configuration or dependencies. *Mechanism choice* concerns how the behaviour is implemented. For example, a verifier may be necessary for a requested feature even if its token-generation algorithm was never specified.

3 Related Work

3.1 From Issues to Implementations

Repository-level issue-resolution benchmarks show that repairs require locating files, following project conventions and responding to execution feedback, not merely producing a plausible code fragment [5, 13, 16]. Research on issue descriptions and issue–PR links also documents variation in task detail and in what those links represent [2, 10, 14]. An issue is therefore useful context for a patch, though it need not be a complete implementation specification.

Studies of coding-agent contributions use public PR records to examine adoption, acceptance and revision [6, 12]. Those records are valuable for studying development activity; they cannot, on their own, establish which operations an issue required or who chose each implementation detail.

3.2 Security and Adversarial Guidance

Prior research finds security weaknesses in model-generated code and documents the influence of surrounding examples and task context [4, 8]. Controlled experiments also demonstrate that malicious issue or bug-report content can steer repair agents toward harmful patches [1, 9]. A related body of work examines instructions injected through retrieved material and agent-tool interfaces [3, 7, 15].

Controlled attack studies begin with an adversarial objective against which success can be measured. Historical PRs lack that ground truth and do not necessarily expose all the information available during implementation.

3.3 The Gap Addressed Here

Research on issue resolution explains how code changes are produced, while controlled attack studies can measure success against a known adversarial objective. Neither establishes how a scanner-flagged operation in a historical PR relates to the linked request. This study examines that relationship using the public development record, without assuming malicious intent or attributing a patch to an agent solely from a marker.

4 Study Design

Figure 1 follows the study from public issue–PR records to scanner findings and changed operations. The main analysis traces those operations and examines available evidence for their implementation (RQ1). Two supporting analyses compare matched workflows (RQ2) and replay selected tasks with altered information (RQ3). They share a historical corpus, but are not parts of one controlled causal experiment.

4.1 Selecting the Workflows

A search for merged PRs linked to GitHub issues (`is:pr is:merged linked:issue`) supplied the candidates. The issue and PR had to belong to the same repository and meet the criteria in Table 1. This yielded 3,809 issue–PR workflows from 723 repositories. Here, a workflow denotes one retained issue–PR relationship, not proof that the issue preceded or instructed every modification.

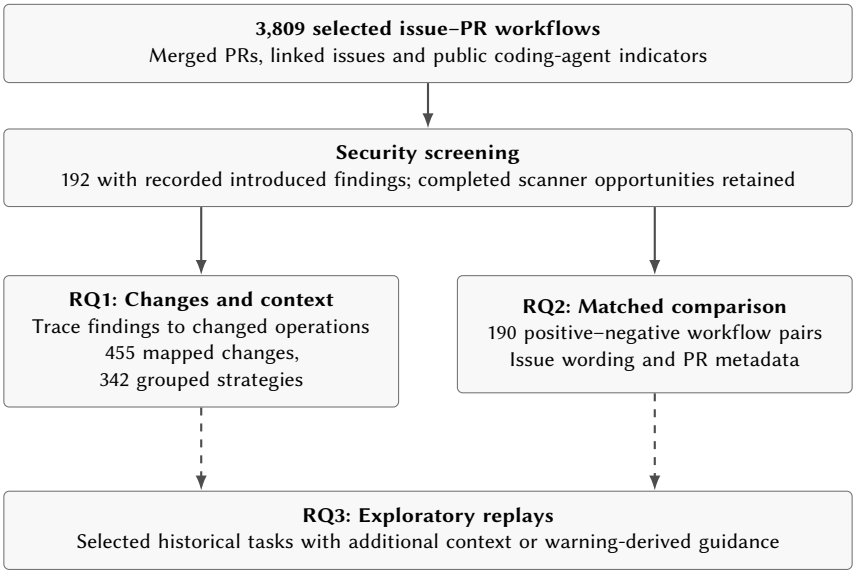


Fig. 1. Study overview. The main analysis traces security findings to changes and their implementation strategies. Matched workflows and selected task replays provide supporting analyses; dashed arrows do not imply random assignment or causality.

Table 1. Selection criteria for the 3,809 workflows. Dates refer to PR merges, not issue creation.

Criterion	Selected population
PR and issue	Merged PR linked to an issue in the same repository
Merge dates	1 November 2025 to 11 August 2026
Repository	JavaScript or TypeScript; non-archived
Popularity	At least 100 stars in the recorded repository metadata
Agent indicator	High-confidence recorded reference to an agent tool, including Codex, Copilot, Cursor, Claude or Devin patterns
Population	3,809 workflows in 723 repositories (609 JavaScript; 3,200 TypeScript)

Indicators appeared in PR authors, branches, descriptions and issue text. For 236 workflows the marker appeared only in issue text, insufficient by itself to establish agent participation in the PR. Hence the term *agent-associated* rather than *agent-authored*. Repository popularity and language come from recorded metadata, not historical reconstructions for each merge date. The cohort is a selected sample, not a random sample of GitHub.

4.2 Security Analysis

Security screening covered source code, dependencies, possible secrets and version downgrades (Table 2).

Table 2. Security checks used to identify candidate conditions for attribution.

Check	Condition examined
CodeQL	Program-analysis warnings, comparing base and PR-head code where applicable
Semgrep	Rules applied to relevant changed source files
OSV and Trivy	Dependency versions associated with published advisories
Gitleaks	Credential-like content on eligible added lines
Dependency-downgrade check	Changes to older dependency versions

Scanner execution was recorded separately from scanner output. Some checks did not apply to a particular repository or change. Where the tools supported a before/after comparison, conditions already present at the PR's merge base were separated from conditions introduced by the PR. Validation of merge-base and dependency transitions corrected the initial scan results. After validation, 192 workflows had retained introduced findings; the remaining 3,617 had none under the recorded checks. An absent result is not a security certificate. For the matched comparison, scanner availability also matters: an unrun CodeQL check cannot be treated as equivalent to a successful scan with no finding. Controls were therefore selected using completed scanner families.

4.3 From Findings to Changed Operations

The analysis links tool output to the patch in three steps. A *scanner finding* is a tool-reported condition. Reports about the same condition are merged into a *canonical finding*, using rules and locations for source code or package, version and advisory identities for dependencies. The *producing change* is the code operation or dependency transition associated with that finding. Several reports may trace to one change.

An attribution required an identifiable changed file or package, an added code location or version transition, and a reason that change would produce the reported condition. Unresolved links were left unmapped. One CodeQL warning about incomplete URL validation, for instance, was traced to the newly added check `pastedText.includes("github.com/")`, rather than attributed to the whole feature patch. This identifies an operation in the diff without deciding exploitability or authorship.

The first annotation group contained 122 scanner-positive workflows with 322 mapped producing changes. An additional 70 workflows contributed 133 mapped changes. Together, these groups contain 455 mapped changes across 191 workflows. In the remaining workflow, 61 findings could not be linked to an exact producing change and were excluded from change-level classification. The detailed finding-type breakdown in the Results concerns only the original 122 workflows; the full 455-change set supports the implementation analysis.

Producing changes were coded by their role in the task. *Direct* changes implement requested behaviour; *supporting* changes contribute through tests, configuration, dependencies or related work. *Unclear* indicates insufficient evidence. These labels describe a change's role, not whether it was justified or whether the issue specified its exact mechanism. The coding included language-model assistance and review corrections; independent human agreement was not established for all labels.

4.4 Grouping and Explaining Implementations

Several changed operations or warnings may stem from one implementation strategy. Changes were therefore grouped within a workflow when they addressed the same subproblem using the

same mechanism. Each group is an *implementation root*. The 455 mapped changes yield 342 roots under the recorded assignments, which organise the evidence rather than establish 342 statistically independent decisions.

To investigate each strategy, the analysis examined issues, discussions, earlier repository code, project instructions, commits, reviews and tooling records. Only material available before the change could serve as a possible precedent. Even then, an earlier implementation pattern shows what was available, not what the implementer actually saw.

The historical classification separated prescribed mechanisms, mechanical tooling consequences and choices among plausible alternatives. For the last category, it also recorded how specifically the public context explained the operation. Change groupings can be traced to preserved records, but the final semantic labels were not retained as a complete independently adjudicated table. These labels describe public explanations, not an agent's private reasoning.

4.5 Comparing Matched Workflows

RQ2 tests whether observable issue and PR characteristics differ between workflows with and without retained scanner findings. From 3,617 scanner-negative candidates, controls were chosen without replacement. Preference went to the same repository, provided the control had completed at least the scanner families available for the positive workflow; cross-repository matches required the same completed families. PRs also had to be within 90 days and approximately fourfold bounds for changed files and lines. The procedure yielded 190 pairs, including 115 within-repository pairs. Two scanner-positive workflows remained unmatched.

Issue wording was coded as a problem statement, a suggested solution or explicit implementation instructions. The comparison also considered issue–PR goal alignment, review activity, timing and agent markers. Each matched pair is one unit of comparison. Percentages come from the recorded historical analysis, whose complete semantic rows and inferential settings were not available for reproduction. Unmeasured differences in task complexity or repository practice remain, so the comparison is descriptive rather than causal.

4.6 Selected Implementation Replays

RQ3 considers selected attempts to reimplement historical tasks with different information. Early pilots varied issue wording or repository context, but their tasks and outcomes differed, so they are kept separate from the main replay series.

The larger warning-informed series used 14 selected historical tasks. Each had three intended conditions: the original issue alone; the issue supplemented with the historical CodeQL warning; and the issue supplemented with a safety requirement derived from that same warning, without quoting the rule identifier. The archive contains 40 patches, and 12 tasks have an output in all three conditions. The comparison uses these 12 tasks, not 40 independent trials. Both enriched conditions depend on information learned after the original warning and are therefore retrospective.

The recorded outputs were compared using scanner alerts and available syntax checks. These measures describe the generated patches; completion of the requested feature was not evaluated as a separate outcome.

4.7 Measures and Evidence Boundaries

For RQ1, the measures are change-level task relation and the recorded explanation category of each grouped mechanism. RQ2 uses matched pairs to compare issue descriptions and PR characteristics. RQ3 uses the tasks with all three replay conditions to compare reported alerts alongside available patch checks. Their populations are kept separate in the Results and the validity discussion (Section 7).

5 Results

5.1 RQ1: How Do the Reported Changes Relate to the Task?

Within the original 122 annotated workflows, 605 canonical findings mapped to 322 producing changes. Task-relation coding classified 91 as direct, 228 as supporting and three as unclear. The breakdown by reported condition appears in Table 3. These are counts of mapped changes, not vulnerable PRs or independent decisions.

Table 3. Task relation of the 322 mapped changes in the original 122-workflow annotation group. The table reports change-level annotation labels.

Reported condition	Direct	Supporting	Unclear	Total
Source-code warning	82	82	2	166
Dependency advisory	3	83	1	87
Dependency downgrade	1	49	0	50
Secret-like string	3	13	0	16
Other dependency change	2	1	0	3
Total	91	228	3	322

The predominance of supporting changes comes largely from dependency findings. Advisories and downgrades account for 132 supporting changes; a package update can also affect transitive versions and lockfiles without a separate choice about each dependency. Source-code warnings show a different pattern, with 82 direct and 82 supporting changes. It would be misleading to interpret the pooled figures as widespread unnecessary source-code work.

A further 70 workflows contributed 133 mapped changes (43 direct and 90 supporting). Across both annotation groups, 455 changes were mapped in 191 workflows; one scanner-positive workflow had no exact change mapping. Table 3 remains restricted to the original 122 workflows because its family breakdown was recorded for that group.

5.2 RQ1: What Explains the Particular Implementation?

Grouping the 455 mapped changes by implementation strategy produced 342 roots. A single operation accounts for 283 roots, while 59 group several related operations. This avoids counting multiple manifestations of a strategy as separate mechanisms; the groups are not thereby statistically independent choices.

The recorded classification distinguishes three explanation types (Table 4). Earlier public material prescribed the exact mechanism for six roots; 103 were classified as mechanical tooling consequences, and 233 as selections among plausible alternatives. For 128 of those 233, the repository context made the operation plausible without explaining why that mechanism was chosen over another. These semantic labels come from the historical analysis and were not independently re-adjudicated.

Table 4. Historical classification of the 342 grouped implementation strategies. These are interpretive labels from the original analysis.

Type of explanation	Roots
Exact mechanism prescribed in earlier public material	6
Mechanical consequence of tooling or generation	103
Selection among plausible implementation mechanisms	233
Total	342

A generated lockfile may contain a flagged dependency change without a separate design choice about that package. Source code that directly serves a feature, by contrast, can still involve a choice among algorithms. In the opening example, the verifier served the requested feature, but the issue did not prescribe the token-generation operation. The earlier repository idiom shows that an alternative existed, not that it was considered.

5.3 RQ2: Differences Between Matched Issues

The matched comparison includes 190 workflow pairs, each containing one scanner-positive workflow and one control with comparable completed scans. Of these, 115 pairs share a repository. Issue wording showed the clearest reported difference (Table 5).

Table 5. Issue wording in 190 historically matched positive–control pairs. The percentages are from the recorded comparative analysis; the 115 same-repository pairs are a subset, not a second experiment.

Issue description	With finding	Matched control
Explicit implementation instructions	63.7%	77.9%
Suggests a solution	17.4%	4.7%
States the problem without a solution	18.9%	17.4%

Scanner-positive issues less often gave explicit implementation instructions and more often suggested a solution without directly instructing its implementation. The 115 same-repository pairs showed a similar contrast: 67.0% of positive issues versus 82.6% of controls included explicit instructions. Whether the issue prescribed the exact flagged mechanism, or whether more detail would prevent a finding, cannot be determined from these comparisons.

Issue–PR goal alignment was also lower in the scanner-positive group, although that contrast weakened within same-repository pairs. No clear reported differences emerged for issue purpose, review activity or issue-to-PR timing.

5.4 RQ3: Observations From Task Replays

An early pilot observed more frequent reuse of an implementation mechanism when repository examples were displayed. The signal did not recur in a stricter bidirectional comparison, partly because the examples could not always be isolated. Those pilots are therefore reported separately from warning-informed replays.

Twelve tasks yielded patches under all three replay conditions. Compared with the original-issue condition, seven had fewer recorded new alerts when the historical CodeQL warning was supplied; four had fewer when given the warning-derived safety requirement. Table 6 reports total alerts for the same twelve tasks.

Table 6. Recorded new-alert totals in the twelve tasks with all three replay conditions. Both added-information conditions were constructed using the historical warning.

Task information supplied	Recorded alerts
Original issue	21
Issue and historical CodeQL warning	10
Issue and warning-derived safety requirement	15

The complete warning was associated with fewer recorded alerts than the warning-derived safety requirement (10 versus 15). This contrast raises a question the replays cannot settle: did the form of the guidance alter the implementation mechanism, or did the resulting patches cover different amounts of the task? One original-issue patch had no recorded alert but failed a syntax check, illustrating why scanner silence is not evidence of a completed implementation.

6 Discussion

6.1 What the Issue Leaves Open

A change may also be essential to a task without its mechanism being prescribed. The Spotify PR included an authentication verifier, but the issue did not specify its token-generation method. The warning concerns that method, not the necessity of the feature.

6.2 What Public Evidence Can and Cannot Explain

A single strategy may produce several mapped changes, particularly when dependency tools propagate updates. Earlier code or explicit guidance can suggest where an implementation came from, although availability is not evidence that the implementer used it. Conversely, failure to find a public precedent leaves open the possibility of private instructions or unexamined context. The route labels describe what is visible, not a count of autonomous mistakes.

6.3 Interpreting the Comparisons

The issue-specificity contrast fits the possibility that open-ended requests leave more implementation details to the developer or agent. But difficult tasks may also elicit different wording and more warnings, while repository conventions can shape both issue reports and patches. Matching improves comparability on recorded context, timing and scanner opportunity. It cannot settle those alternatives, particularly because patch size is itself an outcome of implementation. The matched comparison contains no counterfactual version of the same issue.

The replay contrast suggests a more focused next experiment than simply adding security language to issues. A complete historical warning and a requirement distilled from it were followed by different recorded alert totals. Testing whether that distinction matters would require independent implementations that satisfy the same task, with the form of safety guidance varied deliberately.

6.4 Implications

An accepted PR, a passing test and the absence of scanner warnings answer different evaluation questions. A useful assessment of coding agents would report whether the intended behaviour was delivered, which mechanism was used and what security property was checked. Mechanical changes should be separated from design choices; security outcomes should be compared only across patches with demonstrably comparable functionality.

7 Threats to Validity

7.1 Selection and Measurement

The sample covers merged issue-linked PRs in selected public JavaScript and TypeScript repositories. Unmerged work, private projects and tool use without recognised markers fall outside it. Issue-only markers, in particular, provide weaker evidence of PR participation than an agent author or PR-side marker. The analysis cannot estimate all coding-agent activity or compare verified agent-written code against human-written code.

Scanner coverage differs across tools. Some reported conditions may be false positives, and an absence of findings cannot rule out other defects. Task-relation and provenance coding involved judgment, including language-model-assisted annotation and review corrections. Because the full semantic root decisions were unavailable for an independent recount, their interpretive labels should not be treated as fully adjudicated ground truth.

7.2 Attribution and Comparison

An issue–PR link does not establish the complete instruction history, and earlier repository code does not prove actual exposure to that code. Matching controls on recorded scanner opportunity and observable PR characteristics leaves task and repository differences unmeasured. In addition, portions of the historical matched annotations and inferential settings were unavailable for full reproduction. The comparisons are accordingly interpreted as associations.

The two enriched replay inputs drew on the warning observed in the historical PR. Their outcomes describe selected implementations given hindsight information; they do not establish what an issue author could have anticipated before that warning existed.

7.3 Scope of the Conclusions

The 122-workflow change breakdown, 342 grouped roots, 190 matched pairs and 12 complete replay tasks use different analytical units. None provides a prevalence estimate for agent-associated PRs or GitHub as a whole. Their shared scope is the observable relationship between requests, changed operations and scanner reports, without claims about malicious intent, private reasoning or confirmed exploitability.

8 Conclusion

The issue, the resulting patch and a scanner warning each reveal part of a development decision. Tracing findings to changed operations showed why a simple count of supporting changes can mislead: dependency effects contribute heavily to that count, whereas source-code warnings were evenly divided between direct and supporting work in the original annotation group. Grouping related changes also helps distinguish an implementation mechanism from its repeated manifestations.

Matched workflows differed in issue specificity, and the selected replays produced different alert profiles depending on the information provided. The next question is whether guidance can change an implementation mechanism without changing the intended task. Answering it requires three separate observations: what behaviour was delivered, how it was implemented and which security condition was reported.

References

[1] Simin Chen, Yixin He, Suman Jana, and Baishakhi Ray. 2025. Red Teaming Program Repair Agents: When Correct Patches Can Hide Vulnerabilities. arXiv preprint arXiv:2509.25894. doi:10.48550/arXiv.2509.25894

[2] Cleidson R. B. de Souza, Emilie Ma, Jesse Wong, Dongwook Yoon, and Ivan Beschastnikh. 2024. Revealing Software Development Work Patterns with PR-Issue Graph Topologies. *Proceedings of the ACM on Software Engineering* 1, FSE, Article 106 (July 2024), 22 pages. doi:10.1145/3660813

[3] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. In *Advances in Neural Information Processing Systems*, Vol. 37. Curran Associates, Inc., 82895–82920. doi:10.52202/079017-2636

[4] Yujia Fu, Peng Liang, Amjed Tahir, Zengyang Li, Mojtaba Shahin, Jiaxin Yu, and Jinfu Chen. 2025. Security Weaknesses of Copilot-Generated Code in GitHub Projects: An Empirical Study. *ACM Transactions on Software Engineering and Methodology* 34, 8, Article 218 (2025), 34 pages. doi:10.1145/3716848

[5] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *The Twelfth International Conference on Learning Representations*. https://openreview.net/forum?id=VTF8yNQM66

[6] Hao Li, Haoxiang Zhang, and Ahmed E. Hassan. 2025. The Rise of AI Teammates in Software Engineering (SE) 3.0: How Autonomous Coding Agents Are Reshaping Software Engineering. arXiv preprint arXiv:2507.15003v1. doi:10.48550/arXiv.2507.15003

[7] Yue Liu, Yanjie Zhao, Yunbo Lyu, Ting Zhang, Haoyu Wang, and David Lo. 2026. "Your AI, My Shell": Demystifying Prompt Injection Attacks on Agentic AI Coding Editors. arXiv preprint arXiv:2509.22040v2. doi:10.48550/arXiv.2509.22040

[8] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. 2022. Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions. In *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 754–768. doi:10.1109/SP46214.2022.9833571

[9] Piotr Przymus, Andreas Happe, and Jürgen Cito. 2026. Adversarial Bug Reports as a Security Risk in Language Model-Based Automated Program Repair. In *Proceedings of the 23rd International Conference on Mining Software Repositories (MSR '26)*. Association for Computing Machinery, New York, NY, USA, 212–223. doi:10.1145/3793302.3793352

[10] Xin Tan, Minghui Zhou, and Zeyu Sun. 2020. A First Look at Good First Issues on GitHub. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '20)*. Association for Computing Machinery, New York, NY, USA, 398–409. doi:10.1145/3368089.3409746

[11] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. 2025. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. In *The Thirteenth International Conference on Learning Representations*. https://openreview.net/forum?id=OJd3ayDDoF

[12] Miku Watanabe, Hao Li, Yutaro Kashiwa, Brittany Reid, Hajimu Iida, and Ahmed E. Hassan. 2026. On the Use of Agentic Coding: An Empirical Study of Pull Requests on GitHub. *ACM Transactions on Software Engineering and Methodology* 35, 10 (2026), 1–24. doi:10.1145/3798166

[13] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems*, Vol. 37. Curran Associates, Inc., 50528–50652. doi:10.52202/079017-1601

[14] Zhou Yang, Chenyu Wang, Jieke Shi, Thong Hoang, Pavneet Kochhar, Qinghua Lu, Zhenchang Xing, and David Lo. 2023. What Do Users Ask in Open-Source AI Repositories? An Empirical Study of GitHub Issues. In *2023 IEEE/ACM 20th International Conference on Mining Software Repositories (MSR)*. IEEE, 79–91. doi:10.1109/MSR59073.2023.00024

[15] Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. 2024. InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents. In *Findings of the Association for Computational Linguistics: ACL 2024*. Association for Computational Linguistics, Bangkok, Thailand, 10471–10506. doi:10.18653/v1/2024.findings-acl.624

[16] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. 2024. AutoCodeRover: Autonomous Program Improvement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA '24)*. Association for Computing Machinery, 1592–1604. doi:10.1145/3650212.3680384